pfi

Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

The Agent Loop by Hand

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: the agent loop by hand

Three synthetic bullets, settling 2026-10-02

	coupon	maturity	yield	price	DV01
The week's bond	5.000%	2031-10-02	5.06%	99.737845	0.043625
Bond B	4.250%	2029-10-02	4.40%	99.582714	0.027749
Bond C	6.125%	2036-10-02	6.30%	98.716092	0.072776

Computed by the tools before any request.

Today

1. Day 1's two requests as a loop
2. When a tool fails: run_tools
3. agent_loop, docstring first
4. Two tools, and none
5. Reading a run: trace_table
6. Two ways to stop
7. What each step resends

Every model response in the notebook is a saved response, replayed. A live run can differ.

The rules for agents

What an agent produces: text, every number from a tool

Allowed names

the agent returned

the saved run

the brief states

Never called

the agent's opinion or view

what the agent believes

the agent's advice

On the synthetic book: a description, never a recommendation

Allowed names

DV01 per 100 of face

position DV01 in dollars

the largest DTS contributions in the file

Never called

cheap, rich, or attractive

a trade idea or a recommendation

overweight or underweight

Day 1's two requests as a loop

Send

the whole conversation

Act

run each tool the reply names

Stop

text, or 8 requests

```
for step in range(1, 9):
    reply = client.chat.completions.create(...)
    messages.append(sent)
    if not message.tool_calls:
        break
    messages.append(agentkit.tool_message(...))
```

In the saved run

```
step 1: tool_calls; tool requests ['dv01']

step 2: stop; tool requests []
```

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 949 prompt and 104 completion tokens; saved run recorded 2026-10-08; replayed (no key loaded).

Every agent of the course is these three parts with more tools around them.

When a tool fails

Four bad tool requests, written by hand: what run_tools returns

A tool not in the list	`{"error": "unknown tool 'yield_to_maturity'; the tools are ['dv01', 'price']"}`
Arguments not JSON	`{"error": "the arguments are not JSON: Expecting property name enclosed in double quotes"}`
An argument it does not take	`{"error": "wrong arguments for dv01: dv01() got an unexpected keyword argument 'coupon'"}`
An exception inside it	`{"error": "dv01 failed: ValueError: day is out of range for month"}`

Each failure goes back to the model as a result it can read. A failure is never a number.

agent_loop, docstring first

What it sends, and when it stops

DESK_PROMPT first

temperature 0, seed 42

parallel_tool_calls false

every tool through run_tools

stop "answer": a reply with no tool request

stop "max_steps": after 8 requests

The run it returns, and what reads it

messages, steps, stop

responses, params, exchanges

trace_table: the run as rows

run_usage: requests and tokens

run_line: the line under every run

Day 1's question: the same requests as the hand loop, key for key: yes

A function that sends what you would have sent by hand can replace the hand code.

Two tools, and none

Bond B's price and DV01

tools requested, in order: [price, dv01]

3 requests, one tool per reply

answer: 99.582714 and 0.027749

Bond B's terms, as the question gives them

1 request, no tool request

every number one the question wrote

tools offered are not tools used

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 3 requests, 1,957 prompt and 198 completion tokens; saved run recorded 2026-10-08; replayed (no key loaded).

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 1 request, 572 prompt and 42 completion tokens; saved run recorded 2026-10-08; replayed (no key loaded).

The order came from the model. The saved run fixes the path.

Reading a run: trace_table

What the agent did, step by step

```
0  user  the question
1  assistant  price: arguments; tool_calls
1  tool  {"clean_price_per_100": 99.582714, ...}
2  assistant  dv01: arguments; tool_calls
2  tool  {"dv01_per_100": 0.027749, ...}
3  assistant  the answer; stop
```

6	2	1,957	198
rows	tool requests	prompt tokens	completion tokens

Read what the agent did before what it said. Tool rows cost nothing: they ran on your machine.

Two ways to stop

answer

a reply with no tool request

max_steps

the step limit: 8 requests

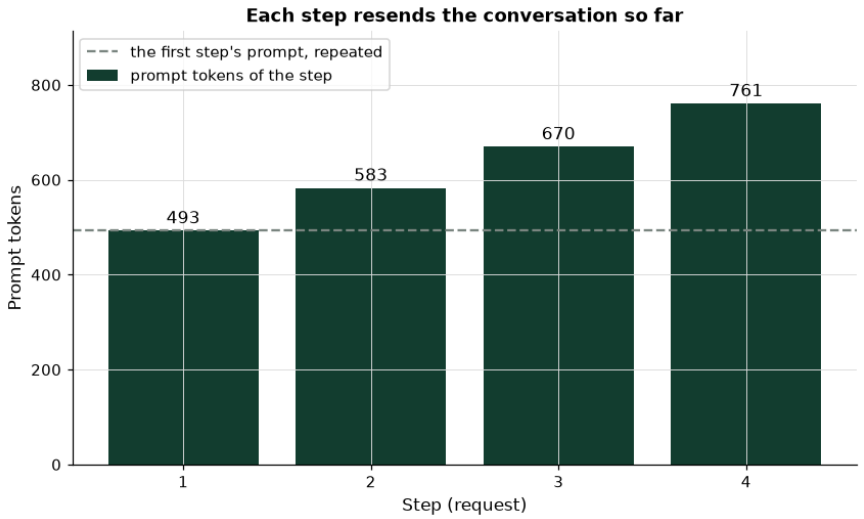
The tests' fake server: the same tool request every time, nothing sent

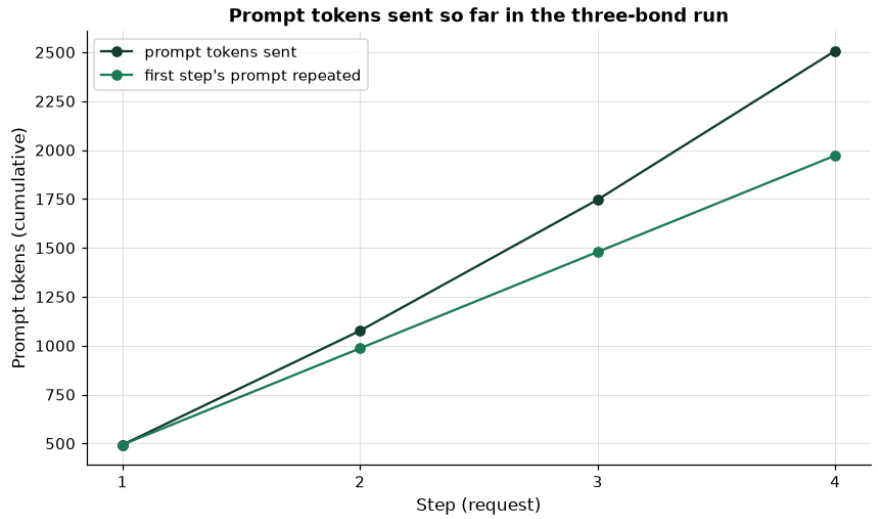
```
stop: max_steps; steps: 8; requests the fake server received: 8
```

the last message: a tool result no request carried back

Tell the two endings apart by the stop reason: a run stopped at the limit has no answer.

What each step resends



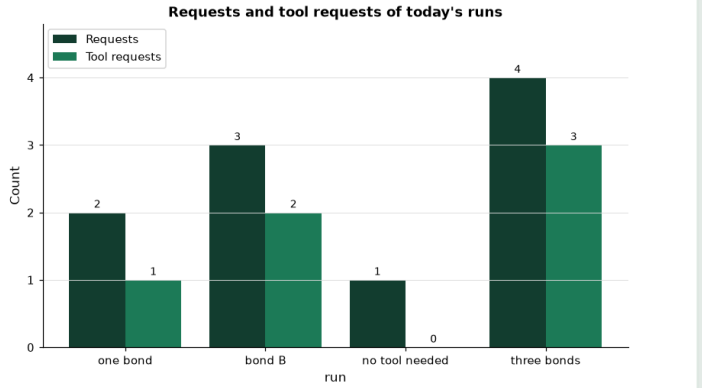


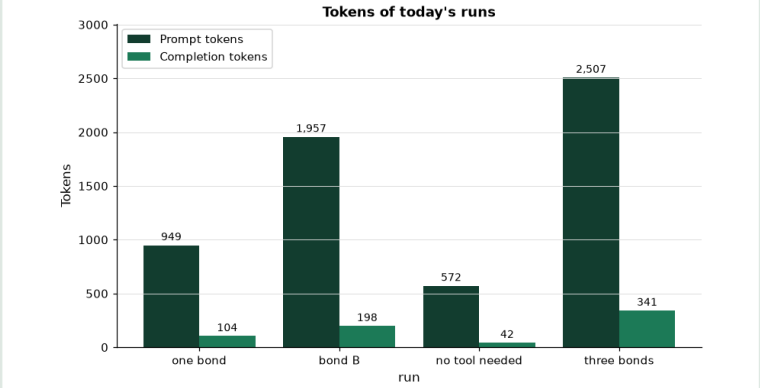
prompt tokens in the run: 2,507; if every step cost the first step's: 1,972; the run: US$0.001548

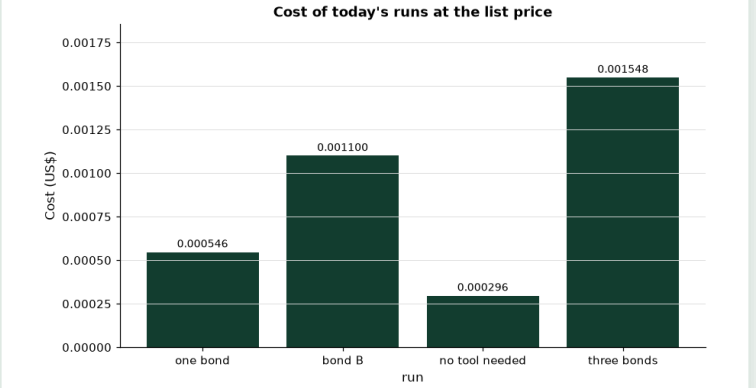
Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 4 requests, 2,507 prompt and 341 completion tokens; saved run recorded 2026-10-08; replayed (no key loaded).

Every request resends the whole conversation: prompt tokens grow faster than steps.

Today's four runs side by side







Run	Requests	Tool requests	Prompt tokens	Completion tokens	Cost (US$)	Stop
one bond	2	1	949	104	0.000546	answer
bond B	3	2	1,957	198	0.001100	answer
no tool needed	1	0	572	42	0.000296	answer
three bonds	4	3	2,507	341	0.001548	answer

A name for the pattern

The arXiv record of 2210.03629

Title	ReAct: Synergizing Reasoning and Acting in Language Models
Authors	seven, the first Shunyu Yao
Posted	6 October 2022; version 3, 10 March 2023
Comment	v3 is the ICLR camera ready version with some typos fixed.

The name given to today's loop: a step from the model, an action by code, the result sent back.

Source, read 8 October 2026: export.arxiv.org/api/query?id_list=2210.03629 (the record, not the paper)

Excel and Git

Excel side by side (tested in Excel)

`=COUNTIF(C2:C7,"price")  (rows that name price)`	2
`=COUNTIFS(B2:B7,"assistant",C2:C7,"price")  (requests)`	1
`=SUM(G2:G7)  (prompt tokens)`	1957
`=SUM(H2:H7)  (completion tokens)`	198
`=SUM(G2:G7)*J2+SUM(H2:H7)*J3  (cost)`	0.0010996

```
run,model,requests,prompt_tokens,completion_tokens,usd,steps,stop,figures_traced,passed
dv01,gpt-4.1-mini-2025-04-14,2,949,104,0.000546,2,answer,,
```

runs.csv: one row per run, committed with agentkit's two files; 7 tests at the start of the day.

The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises, on a fake server. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: while True: and no limit, so a server that keeps asking got 21 requests.

Code that runs is not the same as code that is right.

What you can do now

- Write the agent loop: send, act, stop
- Every tool failure an error result: run_tools
- agent_loop under a step limit of 8
- Read the trace table, arguments included
- Count a run's tokens; price it

Week 1, Day 3 Tools from your toolkit